

Grupo 2 - Javier Tejera, Marcos Bueno y Alejandro Martín

Índice

Índice.....	1
Idea principal:.....	1
Otras ideas (en caso de no gustar la principal).....	1
Base de datos: PostgreSQL.....	1
Backend (Java).....	2
Framework.....	2
Funciones:.....	2
Frontend (React) (Puede cambiar).....	2
Framework.....	2
Bibliotecas útiles:.....	2
Pantallas principales:.....	2
Funcionalidades desarrolladas.....	3
Módulo de stock.....	3
Registro de entradas:.....	3
Alertas.....	3
Historial de movimientos.....	3
Módulo TPV.....	3
Pantalla de venta.....	3
Cierre de caja.....	3
Integración con escáner de código de barras.....	3
Justificación.....	4

Nuestro grupo hará una aplicación de escritorio.

Idea principal:

- Sistema de control de stock con escaneo de códigos de barras + Simulador de caja registradora (TPV) para comercio

Otras ideas (en caso de no gustar la principal)

- Sistema de registro de usuarios + Gestor de archivos top + gestor de contraseñas + carpeta oculta con cifrado y permisos por usuario.
- Aplicación para despliegue de servidores con distintas tecnologías.

Base de datos: PostgreSQL

- **usuarios:** id, nombre, contraseña (hash o alguna otra tecnología).

- **proveedores:** id, nombre, nif, teléfono.
- **categorías:** id, nombre (para clasificar productos).
- **productos:** id, código_barras/qr (único), nombre, descripción, precio_venta, precio_compra, stock_actual, stock_minimo, id_categoria, id_proveedor.
 - Sistema de aviso al usuario de que el stock está por debajo de los mínimos.
- **movimientos_stock:** id, id_producto, tipo (entrada/salida), cantidad, fecha_hora, motivo (compra, venta), id_usuario.
- **ventas:** id, fecha_hora, total_bruto, total_iva, total_netto, forma_pago (efectivo/tarjeta), id_usuario (cajero), cerrada (booleano para cierre de caja).
- **lineas_venta:** id, id_venta, id_producto, cantidad, precio_unitario, subtotal, iva_aplicado.
- **cierres_caja:** id, fecha_apertura, fecha_cierre, total_efectivo_realizado, total_efectivo_esperado, diferencia, id_usuario_apertura, id_usuario_cierre.

Backend (Java)

Framework

Spring Boot (recomendado para crear una API REST de forma rápida).

Funciones:

- CRUD de productos, proveedores, categorías.
- Registrar ventas (crea venta + líneas + actualiza stock).
- Movimientos de stock (entradas, ajustes).
- Consultas (productos por código de barras, stock bajo, historial de ventas).
- Autenticación JWT para proteger las rutas según rol.
- Conexión a BD: driver PostgreSQL.

Frontend (React) (Puede cambiar)

Framework

React con Vite (rápido de configurar aunque por ver).

Bibliotecas útiles:

- React Router para navegación.
- Axios para llamadas a la API (pensando en cuál).
- Material-UI o Tailwind CSS para maquetación (sujeto a cambios).
- React Hook Form + Zod para validación de formularios (usuarios).

Pantallas principales:

- Login.
- Dashboard con resumen (stock bajo, ventas del día).
- Gestión de productos (tabla, búsqueda por código de barras, edición).
- Gestión de proveedores y categorías.

- Pantalla de TPV: similar a una caja registradora con campo de escaneo de código de barras, carrito dinámico, totales actualizados.
- Historial de ventas y cierres de caja.

Funcionalidades desarrolladas

Módulo de stock

CRUD de productos: con validación de código de barras único, carga de imagen opcional (guardada en el servidor firebase como archivo).

Registro de entradas:

Un formulario para seleccionar producto (o escanear código) e introducir cantidad. Se actualiza el stock y se guarda el movimiento en el historial.

Alertas

En el dashboard se muestra una lista de productos con stock por debajo del mínimo a modo de alerta para el usuario.

Historial de movimientos

Tabla con filtros por producto, tipo, fechas.

Módulo TPV

Pantalla de venta

Un campo de texto donde se puede leer el código de barras (con un escáner real o teclado). Al presionar Enter, se busca el producto y se añade al carrito.

Carrito mostrando cada línea con cantidad, precio, subtotal. Se pueden modificar cantidades o eliminar líneas.

Forma de pago (efectivo/tarjeta).

Al finalizar, se guarda la venta en BD y se resta stock.

Cierre de caja

Al iniciar la jornada, el usuario abre caja (se registra fecha y usuario).

Durante la jornada, las ventas se acumulan.

Al cerrar, la app muestra el total esperado y el cajero introduce el efectivo real acumulado. Se guarda la diferencia.

Integración con escáner de código de barras

Simulamos un escáner usando un campo de texto que al recibir un código (ej. "8412345678901") realiza la búsqueda automática.

Está por verse integrar una biblioteca de lectura de códigos de barras desde cámara web

Javier Tejera, Marcos Bueno y Alejandro Martín

(usando quagga.js o html5-qrcode en el frontend).

Justificación

Este proyecto junta la gestión de datos con una interfaz de usuario en tiempo real. La arquitectura propuesta destaca por su escalabilidad, utilizando Spring Boot para garantizar una lógica de negocio mediante una API REST documentada. La integración de hardware (escáneres) y servicios en la nube (AWS/Firebase) hacen más versátil la aplicación.

Además, utilizamos distintas tecnologías vistas en clase y en las empresas. Java, Firebase, Postgres, AWS, etc. Y por supuesto, porque somos chicos de barrio y algunos familiares nuestros tienen empresas pequeñas, pero pagan por dicho software. Como tal no hay opciones libres y totalmente modificables con licencia MIT. Porque nos mola aprender